

Microsoft IaC

Microsoft
Infrastructure as Code

Trainer: DI Thomas Schleich
June 2026

ETC *brainymotion*

1

Hello!

Thank you for joining me today

Instructor: DI Thomas Schleich

Microsoft Cybersecurity Architect
Microsoft Azure Solution Architect
Microsoft 365 Administrator Expert
Microsoft Certified Trainer

Self-employed

Trainings:

- Microsoft 365 & Azure Security
- Azure Administration & Solution Architecting
- PowerShell
- Windows Server & Client for Admins



© Copyright Microsoft Corporation. All rights reserved.

2

Agenda

- IaC Overview
- Introduction IaC Tools
- Authentication and Authorization
- Azure Automation Use Cases I
- DevOps Basics
- Deployment Platforms
- Azure Automation Use Cases II
- Entra ID Automation / MS Graph

3

Infrastructure as Code

Infrastructure as Code (IaC) uses declarative or scripted definitions to provision and manage infrastructure automatically.

Benefits:

- improves consistency, reduces errors,
- speeds deployments, enables version control
- supports scalable, repeatable environments
- faster provisioning and automation
- reduced configuration drift
- better collaboration across teams

5

IaC Tools

- PowerShell
- Azure CLI
- Azure Resource Manager Templates
- Bicep

6

IaC Tools

PowerShell

- Automation and configuration tool built on .NET
- First released: 2006
- Supported on Windows (2 Versions), macOS, Linux, Cloud Shell
- Prefer to use PowerShell 7
- Extendable via modules & scripts

Preparation

- Modules for Azure
 - Az, Az*
- Modules for Entra ID
 - Microsoft.Graph
 - Microsoft.Entra

```
Find-PSResource -Name Az
Install-PSResource -Name Az -Scope CurrentUser

Install-PSResource -Name Microsoft.Graph
Install-PSResource -Name Microsoft.Entra
```

7

IaC Tools

Azure CLI

- Cross-platform command-line tool for Azure
- First released: 2017 (CLI 2.0)
- Supported on Windows, macOS, Linux, Cloud Shell
- Extendable via CLI extensions and scripts
- Installation
 - In Windows use winget.exe
 - In macOS and Linux use your preferred method

8

IaC Tools

Azure Resource Manager Template

- JSON-based declarative IaC
- Defines desired state of Azure resources
- Supports parameters, variables, modularization

Bicep

- Domain specific language (DSL) for Azure IaC
- Cleaner syntax than ARM JSON and less code
- Modular, type-safe, compiles to ARM

9

Deployment platform

- Deployment Stacks
- Azure Automation Account
- Azure Function
- GitHub Actions
- Azure DevOps Pipelines

10

Authentication & Authorization

- Authentication
 - Process of verifying who you are
- Authorization
 - Determines what you are permitted to access after authentication
- Who
 - User
 - Application
 - Device

11

Authentication - User

- interactive sign in
- automatic sign in sometimes possible but not recommended
- requirements: User Account in Entra ID, MFA configuration
- Flow:
 - OAuth 2.0 Authorization Code Flow
 - result: access token & refresh token
 - more: <https://learn.microsoft.com/en-us/entra/identity-platform/v2-oauth2-auth-code-flow>

12

Authentication - App

- interactive sign in possible
- automatic sign in
- requirements: App registration / Service principal, certificate/secret
- Flow:
 - OAuth 2.0 Client Credentials Grant Flow
 - result: access token (contains app's permission and scope)
 - more: <https://learn.microsoft.com/en-us/entra/identity-platform/v2-oauth2-client-creds-grant-flow>

13

Demonstration

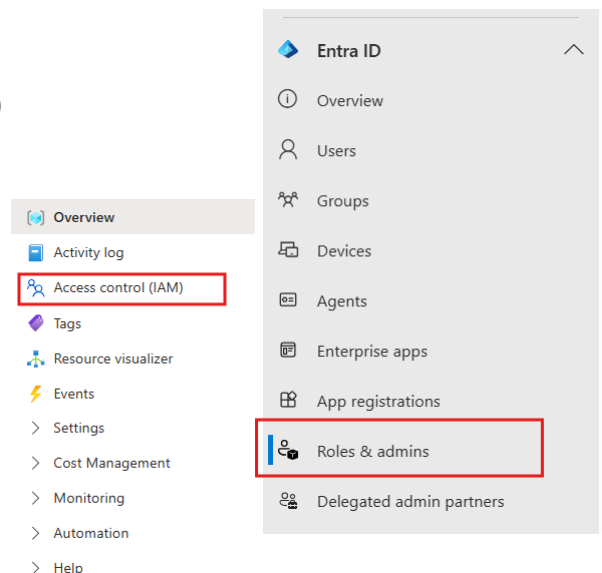
- User sign in, access token
- App/SP sign in, access token

14

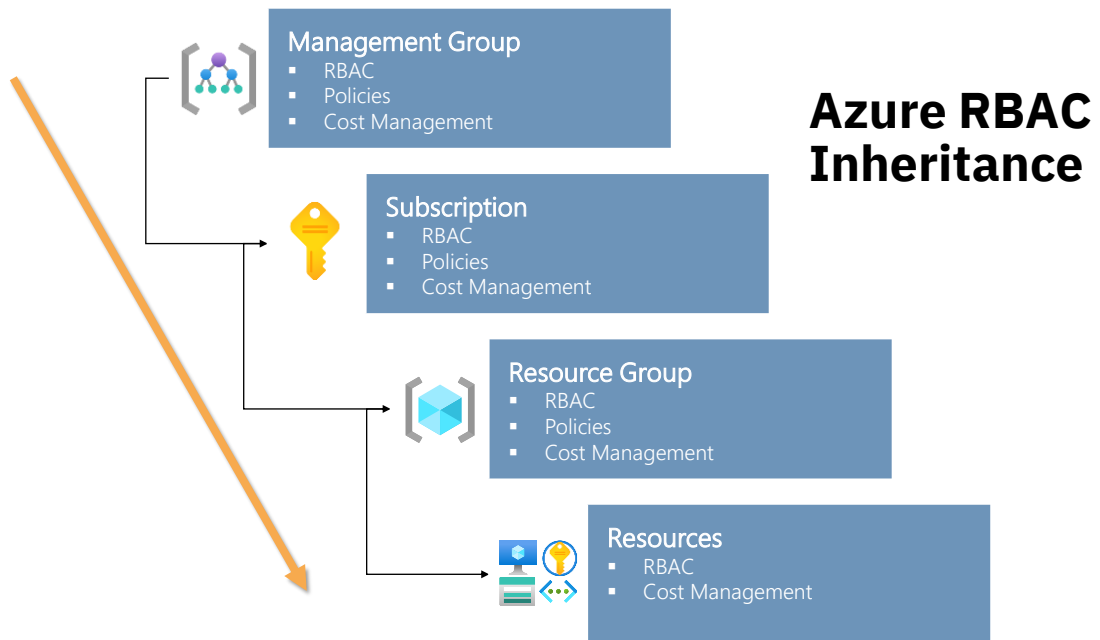
Source: [1] <https://learn.microsoft.com/en-us/azure/role-based-access-control/custom-roles>
 [2] <https://learn.microsoft.com/en-us/entra/identity/role-based-access-control/custom-create-tabs-admin-center>
 [3] <https://learn.microsoft.com/en-us/entra/identity/role-based-access-control/custom-available-permissions>

Authorization

- Role Based Access Control (RBAC)
 - Entra ID: Roles & admins
 - Azure: Access control (IAM)
- Roles
 - Built-in
 - Custom
 - Azure more: [1]
 - Entra ID more: [2] [3](!)
- Hierarchy/Inheritance
 - Entra ID: Admin Units
 - Azure: next slide



15



16

Demonstration

- Azure RBAC
- Azure RBAC Inheritance
- Azure RBAC custom role
- Entra ID RBAC

17

Authorization - beyond RBAC

The RBAC system often grants too many or too few permissions. Therefore, you could use **API Permissions**.

API Permissions

- configured in Entra ID portal
- application vs. delegated permissions
- mostly used for SPs

18

Authentication PowerShell

- Interactive Logon

```
Connect-AzAccount
Connect-AzAccount -AccountId <yourUPN>
Connect-AzAccount -Tenant <yourTenantID>
```

- SP Logon

```
Connect-AzAccount -ServicePrincipal `
  -Credential $credential `
  -TenantId '<yourTenantID>'

Connect-AzAccount -ServicePrincipal `
  -ApplicationId '<yourAppID>' `
  -CertificateThumbprint $cert.Thumbprint `
  -TenantId '<yourTenantID>'
```

- Managed Identity Logon

```
Connect-AzAccount -Identity
```

19

Authentication Azure CLI

- Interactive Logon

```
az login
az login --username <username>
az login --tenant <tenant>
```

- SP Logon

```
az login --service-principal -u <appId> -p <password> --tenant <tenantId>
az login --service-principal -u <appId> --certificate /path/to/cert.pem --tenant <tenantId>
```

- Managed Identity Logon

```
az login --identity
```

20

Demonstration

- Authentication PowerShell + Entra ID
- Authentication Azure CLI + Entra ID
- Find API permissions/scopes in access token

21

ARM Templates



22

Azure Resource Manager Templates (ARM)

ARM Templates are **JSON-based declarative files** used to define Azure infrastructure as code. They describe the **desired state** of resources, so deployments are **repeatable**, **idempotent**, and **auditable**.

- **Declarative model:** specify *what* to deploy, not how.
- **Idempotent deployments:** reapplying the template yields the same result.
- **JSON format:** machine-readable and supported natively by Azure.

23

Azure Resource Manager Templates (ARM)

Why ARM templates matter?

- ARM Templates are a **native, first-class IaC option in Azure**, essential for enterprise governance, automation, and compliance. They integrate with Azure features like **policies, role assignments**, and **template specs**, enabling controlled, auditable infrastructure lifecycles.
- **Use cases**
 - Automated environment provisioning (dev/test/prod)
 - Policy-driven governance and compliance
 - Repeatable CI/CD deployments and disaster recovery

24

ARM Template Authoring

- Visual Studio Code
- Extension: Azure Resource Manager (ARM) Tools
 - maintenance mode
 - no IntelliSense or Snippets anymore
- Visual Studio
 - IntelliSense and Snippets still working at the moment

25

ARM Templates - Structure

```
{
  "$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentTemplate.json#",
  "contentVersion": "1.0.0.0",

  Select or create a parameter file...
  "parameters": { },

  "functions": [ ],

  "variables": { },

  "resources": [ ],

  "outputs": { }
}
```

26

ARM Templates - Parameters

types:

- string
- int
- bool
- array
- object

parameters without
"defaultValue" are
mandatory

```
"parameters": {
  "iacLocation": {
    "type": "string",
    "metadata": {
      "description": "The name of the location."
    },
    "defaultValue": "AustriaEast",
    "allowedValues": [
      "AustriaEast",
      "AustriaWest",
      "CentralEurope",
      "NorthEurope",
      "WestEurope"
    ]
  },
},
```

27

ARM Templates - Parameters

additional attributes

- "maxLength"
- "maxValue"
- "minLength"
- "minValue"

```
"parameters": {
  "iacLocation": {
    "type": "string",
    "metadata": {
      "description": "The name of the location."
    },
    "defaultValue": "AustriaEast",
    "allowedValues": [
      "AustriaEast",
      "AustriaWest",
      "CentralEurope",
      "NorthEurope",
      "WestEurope"
    ]
  }
},
```

28

ARM Templates - Variables

- Variable content: simple value, result of functions
- Functions are available only between "[and]"

```
"variables": {
  "varLocation": "Austria East",
  "varSASName_A": "[parameters('iacSASName')]",
  "varSASName_B": "[concat(parameters('Prefix'), parameters('iacSASName'), 'xyz')]"
},
```

- Usage

```
"outputs": {
  "varLocation": {
    "type": "string",
    "value": "[variables('varLocation')]"
  },
}
```

29

ARM Templates - Variables

```
"variables": {
  "varArray": [
    "Value1",
    "Value2",
    "Value3"
  ],
  "varObject": {
    "VMSize": "Standard_DS1_v2",
    "VMName": "iac-VM-1",
    "Hostname": "VM-1"
  }
},
```

```
"outputs": {
  "varArray": {
    "type": "array",
    "value": "[variables('varArray')]"
  },
  "varArrayPosition0": {
    "type": "string",
    "value": "[variables('varArray')[0]]"
  },
  "varArrayLength": {
    "type": "int",
    "value": "[length(variables('varArray'))]"
  },
  "varObject": {
    "type": "object",
    "value": "[variables('varObject')]"
  },
  "varVMHostname": {
    "type": "string",
    "value": "[variables('varObject').Hostname]"
  },
}
```

30

ARM Templates - Output

- returned values/information from deployment
- "outputName": { "type": "string|int|bool|object|array", "value": "..." }

```
"varLocation": {
  "type": "string",
  "value": "[variables('varLocation')]"
},
```

- Usage:
 - from linked templates e.g.:
"[reference('<deploymentName>').outputs.<propertyName>.value]"
 - Shell:

```
$deployment = Get-AzResourceGroupDeployment `
    -ResourceGroupName '<yourResourceGroupName>' `
    -Name '<yourDeploymentName>'
$deployment.Outputs.varLocation.value
```

31

ARM Templates - Dependency

- **"dependsOn" attribute**
 - if resource A depends on resource B, both must be defined in the same template.
 - resource B in array of "dependsOn"
- **"[resourceId()]" function**
 - determines the resource id of an existing resource in same resource group
 - cannot be used in "dependsOn" attribute
- **"[extensionResourceId()]" function**
 - determines the resource id of an existing resource outside the current resource group/subscription
 - cannot be used in "dependsOn" attribute

32

Source: <https://learn.microsoft.com/en-us/azure/azure-resource-manager/templates/linked-templates?tabs=azure-powershell>

ARM Templates - Modularization

'To deploy complex solutions, you can break your Azure Resource Manager template (ARM template) into many related templates and then deploy them together through a main template. The related templates can be separate files or template syntax that's embedded within the main template.'

Types:

- Nested template
- Linked template

33

Source: <https://learn.microsoft.com/en-us/azure/azure-resource-manager/templates/linked-templates?tabs=azure-powershell#expression-evaluation-scope-in-nested-templates>

ARM Template - Modularization

'When using a nested template, you can specify if template expressions are evaluated within the scope of the parent template or the nested template. The scope determines how parameters, variables, and functions like [resourceGroup](#) and [subscription](#) are resolved.'

expressionEvaluationOptions Values:

- outer (default)
- inner
 - using parameters inside of nested deployments requires "inner"

```
"resources": [
  {
    "type": "Microsoft.Resources/deployments",
    "apiVersion": "2019-10-01",
    "name": "nestedDeployment",
    "properties": {
      "expressionEvaluationOptions": {
        "scope": "inner"
      },
      "mode": "Incremental",
```

34

ARM Template - Deployment

- PowerShell

```
Get-command -Module Az.Resources -Noun *Deployment
```

— Verbs: Test, New, Get, Remove, Stop

- Azure CLI

```
az deployment --help
```

— Subgroups: tenant, mg, sub, group, operation

— Commands: validate, create, list/show, delete, cancel

- Examples

```
Test-AzResourceGroupDeployment -ResourceGroupName '<yourResourceGroupName>' `
  -TemplateFile '.\VNet.bicep'
```

```
az deployment group validate -g "<yourResourceGroupName>" `
  --name DeploymentName `
  --template-file .\VNet.bicep
```

35

ARM Template Reference

- Documentation
<https://learn.microsoft.com/en-us/azure/azure-resource-manager/templates/>
- Create templates
<https://learn.microsoft.com/en-us/azure/azure-resource-manager/templates/syntax>
- Template functions
<https://learn.microsoft.com/en-us/azure/azure-resource-manager/templates/template-functions>
- Resource reference
<https://learn.microsoft.com/en-us/azure/templates/>

36

Demonstration

- Authoring an ARM Template
- parameters, variables, output
- Test Template
- Deploy Template

37

Bicep



38

Bicep

Bicep is a **domain-specific language (DSL)** created by Microsoft to simplify authoring Azure infrastructure as code. It provides a clean, declarative syntax that compiles directly into ARM JSON, making deployments **repeatable**, **idempotent**, and easier to maintain.

- **Simplified authoring:** far cleaner than raw ARM JSON
- **Native Azure support:** fully integrated with ARM
- **Readable & modular:** supports modules, types, and IntelliSense
- **Compiles to ARM:** ensures compatibility with all Azure resources

39

Bicep

Bicep is the **recommended IaC language for Azure**, offering a modern, maintainable alternative to ARM Templates while preserving full Azure compatibility. It improves developer productivity, reduces template complexity, and supports enterprise-grade automation and governance.

- Use cases:
 - **Modular, reusable IaC architectures**
 - **Enterprise deployments with strong governance**
 - **CI/CD pipelines using Azure CLI or DevOps**
 - **Migration from ARM JSON to cleaner Bicep code**

40

Bicep - Authoring & Using

- Bicep CLI `winget.exe install --id Microsoft.Bicep`
 - compiling/decompiling
 - analyzing
 - used by Azure CLI, VSC
- Visual Studio Code + Bicep Extension
- <https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/install>

41

Bicep - Structure

targetScope

- subscription
- resourceGroup
- tenant
- managementGroup

```
targetScope = 'resourceGroup'

param ...

var ...

resource ...

output ...
```

42

Documentation: <https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/parameters#use-decorators>

Bicep - Parameters

@<decorator>(<argument>)

param <Parametername> <Type> = <defaultValue>

- defaultValue is optional

@decorators

- description()
- maxLength()
- maxValue()
- minLength()
- minValue()
- allowed([])
- secure()

```
@description('The prefix for the resources to be created.')
@maxLength(4)
param iacPrefix string = 'iac-'

@description('The location where the resources will be deployed.')
@allowed([
  'AustriaEast'
  'AustriaWest'
  'GermanyNorth'
  'GermanyWestCentral'
])
param iacLocation string = 'AustriaEast'
```

43

Documentation: <https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/file>

Bicep - Variables

@<decorator>(<argument>)
var <varName> <type> = <varValue>

```
param iacLocation string = 'AustriaEast'

var instanceCount int = 3
var NSGName string = 'iacNSG-Default'
var isProduction bool = true
var tags object = {
  environment: 'dev'
  costCenter: 'HR'}
var subnets array = ['subnet1', 'subnet2']

output iacPrefixOutput string = iacPrefix
output iacLocationOutput string = iacLocation
output instanceCountOutput int = instanceCount
output NSGNameOutput string = NSGName
output isProductionOutput bool = isProduction
output tagsOutput object = tags
output firstTagOutput string = tags.environment
output subnetsOutput array = subnets
output firstSubnetOutput string = subnets[0]
```

44

Documentation: <https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/file>

Bicep - Variables

@<decorator>(<argument>)
var <varName> <type> = <varValue>

decorators

- @description()
- @export()

```
param iacLocation string = 'AustriaEast'

var instanceCount int = 3
var NSGName string = 'iacNSG-Default'
var isProduction bool = true
var tags object = {
  environment: 'dev'
  costCenter: 'HR'}
var subnets array = ['subnet1', 'subnet2']

output iacPrefixOutput string = iacPrefix
output iacLocationOutput string = iacLocation
output instanceCountOutput int = instanceCount
output NSGNameOutput string = NSGName
output isProductionOutput bool = isProduction
output tagsOutput object = tags
output firstTagOutput string = tags.environment
output subnetsOutput array = subnets
output firstSubnetOutput string = subnets[0]
```

45

Documentation: <https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/file>

Bicep - Variables

@<decorator>(<argument>)
var <varName> <type> = <varValue>

Construction

- functions
- interpolation

```
@maxLength(4)
param iacPrefix string = 'iac-'

var vnetNameA string = toLower('IAC-VNET-A')
var vnetNameB string = '${iacPrefix}VNet'
```

46

Bicep - Resources

- attributes depend on the resource type

```
resource symbolicName '<ResourceProvider>/<ResourceType>@<apiVersion>' = {
  name: '<resourceName>'
  location: '<location>'
  properties: {}
}
```

47

Bicep - Conditions

Conditional deployments

- resource symbolicName <resourceType> = if (condition) { ... }

```
resource nic 'Microsoft.Network/networkInterfaces@2025-05-01' = if (isProduction) {
  name: 'iac-nic'
```

```
resource nic 'Microsoft.Network/networkInterfaces@2025-05-01' = if (DeployNIC == 'yes') {
  name: 'iac-nic'
```

- Operator ?
 - "if - then - else"

```
output nicDeploymentOutput string = (DeployNIC == 'yes') ? 'NIC will be deployed' : 'NIC will not be deployed'
```

Source: <https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/loops>

Bicep - Loops

'... to define multiple copies of a resource, module, variable, property, or output.'

- [for <index> in range(<startIndex>, <endIndex>): { ... }]

```
param LAWSCount int = 2

resource iac_workspaces 'Microsoft.OperationalInsights/workspaces@2023-09-01' = [
  for i in range(0, LAWSCount): {
    name: 'iac-workspace-${i+1}'
```

- [for <item> in <array> : { ... }]

```
param LAWSNAMES array = ['iacLAWS-1', 'iacLAWS-2']

resource iac_workspaces 'Microsoft.OperationalInsights/workspaces@2023-09-01' = [
  for LAWSName in LAWSNAMES : {
    name: LAWSName
```


Source: <https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/loops>

Bicep - Loops

- [for (<item>, <index>) in <array>: { ... }]
- [for <item> in <array> : if (<condition>) { ... }]
- [for <item> in <array> : if (<item.property>) { ... }]

50

Documentation: <https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/modules>

Bicep - Modules

- Modules are bicep files used by another one
- are local files
- no repository, but container registry

```
module AutomationAccount './ModuleAutoAccount.bicep' = {
  name: 'iac-AutomationAccount-Module'
  params: {
    AutomationAccountName: 'iac-AutomationAccount'
  }
  dependsOn: []
}
```



```
param AutomationAccountName string = 'iac-AutomationAccount'

resource AutoAccount 'Microsoft.Automation/automationAccounts@2024-10-23' = {
  name: AutomationAccountName
  location: resourceGroup().location
  properties: {
    sku: {
      name: 'Free'
    }
  }
}
```

51

Bicep - Dependency

- "dependsOn" attribute
 - if resource A depends on resource B, both must be defined in the same template.
 - resource B in array of "dependsOn"
 - often not necessary
- existing resources
 - declaration **resource** with argument **existing**
 - resource <name> <Type@Version> existing = { }

```
resource existingVNet 'Microsoft.Network/virtualNetworks@2025-05-01' existing = {
  name: 'iac-VNet-Spoke-2'
  scope: resourceGroup('RG-Network-LandingZone')
}
```

- **scope** necessary if resource exist not in same resource group as deployment

52

Bicep Reference

- Documentation
 - <https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/>
- File structure and syntax
 - <https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/file>
- Resource reference
 - <https://learn.microsoft.com/en-us/azure/templates/>

53

Azure Deployments I

- Deployment scripts
- VM custom script extension
- VM shutdown
- VM App installation
- Key Vault integration

54

Deployment I - Deployment Scripts

Azure Deployment Scripts let you run PowerShell or Bash inside an ARM/Bicep deployment to perform tasks that IaC alone cannot handle — such as calling REST APIs, running CLI commands, or performing custom configuration steps.

Why They Exist

- Some deployment logic is too dynamic or conditional for ARM/Bicep
- Useful when you need to orchestrate external systems or perform post-deployment tasks
- Ensures everything runs as part of the deployment pipeline, not manually afterward

55

Deployment I - Deployment Scripts

- Key Capabilities
- Run PowerShell or Bash in a managed container
- Access Azure resources using a managed identity
- Output values back into the deployment
- Store logs automatically in a storage account
- Support for inline scripts or external script files
- Typical Use Cases
- Registering providers or features
- Creating resources not supported by ARM/Bicep
- Running az CLI commands
- Seeding databases or initializing configurations
- Calling custom APIs during deployment

56

Deployment I - Deployment Scripts



Storage
Account



Managed
Identity



Azure
Container
Instance

57



58

Git — Distributed Version Control System

Git is a **distributed version control system (DVCS)** designed to track changes in source code, support collaboration, and maintain complete project history. Every developer has a full copy of the repository, enabling fast, reliable, and offline-capable workflows.

Key points:

- Created by Linus Torvalds in 2005
- Fully distributed architecture
- Tracks changes, branches, merges, and history
- Foundation of modern DevOps and CI/CD workflows

60

Benefits of Git

- **Fast and efficient:** Optimized for large codebases and frequent operations
- **Distributed model:** Every developer has a full local repository and history
- **Powerful branching:** Lightweight branches enable safe experimentation
- **Reliable merging:** Advanced merge algorithms support complex workflows
- **Strong data integrity:** SHA-1 hashing protects against corruption
- **Open source:** Free, transparent, and widely supported
- **Ecosystem integration:** Works seamlessly with GitHub, GitLab, Azure DevOps, Bitbucket
- **Automation-friendly:** Ideal for scripts, DevOps pipelines, and tooling

61

Git Internal Architecture



Key concepts:

- Working Directory: your actual files
- Staging Area: selected changes for the next commit
- Local Repository: full history stored on your machine
- Remote Repository: shared server for collaboration

62

Git Object Model

Git stores everything as **content-addressed objects**, each identified by a SHA-1 hash.

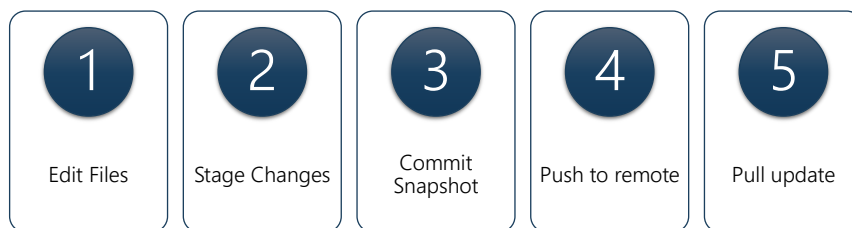
Object types:

- **Blob:** file content
- **Tree:** directory structure
- **Commit:** snapshot + metadata
- **Tag:** human-readable reference to a commit

Key idea: Git does *not* store diffs — it stores **snapshots**, making it fast and reliable.

63

Git Workflow Summary



Benefits:

- Full local history
- Offline work possible
- Fast branching and merging
- Strong integrity via SHA-1
- Works with GitHub, GitLab, Azure DevOps, Bitbucket

64

Git - Staging

Staging is the step where you **prepare** changes before you commit them. Git calls this area the **Staging Area** or **Index**.

Why staging is important

- **Group related changes** into clean commits
- **Exclude accidental edits**
- **Split big work into logical pieces**
- **Review exactly what will be committed**
- **Build a readable project history**

65

Git - Commit

git commit **creates a permanent snapshot** of the changes you staged. It's the moment where your work becomes part of your project's history. Think of it like pressing **Save + Version** at the same time.

When you commit, Git:

- takes everything in the **staging area**
- creates a **snapshot** of those files
- stores it in the **local repository**
- assigns it a unique **SHA-1 hash**
- attaches metadata: author, timestamp, commit message, parent commit(s)

This snapshot becomes a new point in your project's timeline.

66

Git - Installation & first steps

- <https://git-scm.com>

- multiple platforms

- Windows:

```
winget.exe search git
winget.exe show --id Git.Git
winget.exe install --id Git.Git
winget.exe upgrade --id Git.Git
```

- macOS: brew

- Linux: distribution-dependent

<https://git-scm.com/install/linux>

- current version

```
git --version
```

- help

```
# help in Shell
git help
git -h
git command --help
git command -h

# Show help in Browser
git help git
git help anycommand
```

67

Git - Initialization

- start

- Get-Childitem -Force

- .gitignore

https://git-scm.com/docs/gitignore#_pattern_format

```
test*      # Ignore all files that start with "test" in the directory
           # where the .gitignore file is located

docu/      # Ignore the entire "docu" directory and all its contents

docu/*.pdf # Ignore all PDF files in the "testing" directory

/*         # Ignore all files with any extension
           # of that directory in which the .gitignore file is located

**/docu*   # Ignore all files and directories that start with "docu" in any directory

**/docu/   # Ignore all directories named "documentation" and their contents in any directory
```

68

Git - Staging

- adding changes to staging area

```
git add file1.txt  git add .  git add -A
```

- what is staged

```
git status
```

- unstage something

```
git restore
```

69

Git - Commit

- a message must be provided
- only staged files are committed

```
git commit -m "Commit message"
```

- Get information of commits

```
git log
git log --oneline
git log --graph
git log file1.txt
```

70

Git - UnCommit - Oops, I committed to early 😞

- Multiple use cases:

```
git reset HEAD~1      # Unstage the last commit,
                      # keeping the changes in
                      # the working directory

git reset --hard HEAD~1 # Reset to the previous commit,
                      # discarding all changes in
                      # the working directory

git reset --soft HEAD~1 # Reset to the previous commit,
                      # keeping all changes in the
                      # staging area and working directory
```

71

Git - UnCommit

- 'git revert' moves HEAD to last commit without deleting commits in history

```
git revert -n HEAD
git commit -m "Revert last commit"
```

72

Azure Functions



73

Azure Functions

Azure Functions is a serverless compute service that runs small pieces of code without requiring you to manage servers or infrastructure. You write the logic, define the trigger, and Azure handles execution, scaling, and availability automatically. Functions are ideal for event-driven workloads, automation, lightweight APIs, and background processing.

Key Points:

- **Serverless model:** No servers, VMs, or containers to manage
- **Event-driven:** Triggered by HTTP, queues, timers, storage events, Event Grid, etc.
- **Pay-per-execution:** No cost when idle; consumption-based billing
- **Multi-language support:** C#, JavaScript/TypeScript, Python, Java, PowerShell
- **Deep Azure integration:** Easy bindings to Storage, Service Bus, Cosmos DB, Event Hub
- **Local development:** Build and test locally before deploying

74

Azure Functions - Benefits

- **Infrastructure abstraction:** No need to provision or maintain compute resources
- **Perfect for IaC:** Functions can be fully defined and deployed using Bicep, ARM, Terraform, or GitHub Actions
- **Consistent, repeatable deployments:** IaC ensures identical environments across dev, test, and production
- **Event-driven automation:** Ideal for IaC workflows like resource cleanup, policy enforcement, or scheduled tasks
- **Modular microservice pattern:** Each function can be deployed independently as part of a larger IaC-managed system

75

Azure Functions - Supported Languages

Fully Supported

- **C#** (enterprise workloads)
- **JavaScript** / TypeScript (Runs on Node.js)
- **Python** (automation, data processing, AI workflows)
- **Java** (enterprise workloads)
- **PowerShell** (*Ideal for automation and operational tasks*)
- Custom handler
 - any language by hosting your own language service
 - Go, Rust, PHP, Ruby, ...

76

Azure Functions - Input Bindings

Input bindings allow an Azure Function to automatically receive data from external services without writing connection or retrieval code. Azure handles the integration, so the function simply receives the data it needs as parameters.

Examples:

- **Blob Storage:** Read file contents when a blob is created
- **Queue Storage:** Receive messages from a queue
- **Cosmos DB:** Fetch documents based on a query
- **Event Hub:** Consume event stream data
- **HTTP Input:** Receive request body or headers

77

Azure Functions - Input Bindings

Input bindings allow an Azure Function to automatically receive data from external services without writing connection or retrieval code. Azure handles the integration, so the function simply receives the data it needs as parameters.

Key Points:

- **No boilerplate code:** Azure retrieves the data for you
- **Declarative configuration:** Defined in function.json or attributes
- **Supports many Azure services:** Storage, Cosmos DB, Service Bus, ...
- **Automatic deserialization:** Data is passed directly into your function
- **Event-driven:** Function triggers and input bindings often work together
- **Secure integration:** Uses managed identities and connection settings

78

Azure Functions - Output Bindings

Output bindings allow a function to send data to external services without manually creating clients or writing integration code. The function simply returns or sets an output value, and Azure delivers it to the target service.

Examples:

- **Queue Storage:** Add a message to a queue
- **Blob Storage:** Write files or logs
- **Cosmos DB:** Insert or update documents
- **Event Hub:** Publish events
- **Service Bus:** Send messages to topics or queues

79

Azure Functions - Output Bindings

Output bindings allow a function to send data to external services without manually creating clients or writing integration code. The function simply returns or sets an output value, and Azure delivers it to the target service.

Key Points:

- **Simplifies integrations:** No SDK or connection code needed
- **Multiple outputs supported:** A function can write to several services
- **Declarative configuration:** Defined in function.json or attributes
- **Automatic serialization:** Azure formats the output for the target service
- **Ideal for automation:** Write logs, queue messages, or database entries easily
- **Works with IaC:** Bindings are fully definable in Bicep/ARM/Terraform

80

Azure Functions - Files

File	Purpose	Who Uses It	Key Settings / Notes
host.json	Global configuration for the Functions host runtime . Controls behavior of all functions in the app.	Azure Functions runtime (local + Azure)	Logging, retry policies, extension settings, durable functions config, HTTP settings. Example: "version": "2.0"
local.settings.json	Local-only configuration for development. Stores secrets, connection strings, and environment variables.	Local Functions host only (NOT deployed to Azure)	Contains AzureWebJobsStorage, FUNCTIONS_WORKER_RUNTIME, custom app settings. Should be excluded from source control.
requirements.psd1	Defines PowerShell module dependencies for your Function App. Modules listed here are automatically installed in Azure.	Azure Functions PowerShell worker	Add modules like Az.Accounts, Az.Resources, or custom modules. Uses PowerShell module manifest syntax.
profile.ps1	Runs once per Function App instance before any function executes. Used for initialization.	PowerShell worker (startup script)	Good for: importing modules, setting environment variables, connecting to services, caching tokens. Not per-function.
extensions.csproj	Defines .NET extensions used by the Functions host (even for PowerShell apps). Required for triggers/bindings.	Functions host build system	Used to add extensions like Event Hub, Service Bus, Durable Functions. Restored automatically via func extensions install.

81

Demonstration

- Create Azure Functions App
- Prepare local environment
- Create and test function
- Deploy function
- Test function in production

82



Azure Automation Account



83

Azure Automation Account

An Azure Automation Account is a cloud service that provides a centralized platform for automating operational tasks across Azure and hybrid environments. It allows you to run PowerShell or Python runbooks, manage configuration at scale, automate patching, and orchestrate complex workflows without manual intervention. It acts as a control center for operational automation, governance, and configuration management.

Key Capabilities:

- **Runbooks:** Automate tasks using PowerShell or Python scripts
- **Process Automation:** Schedule, trigger, and orchestrate workflows
- **Configuration Management:** Track and enforce desired state using DSC
- **Credential & Asset Management:** Securely store variables, credentials, certificates, and connections

84

Azure Automation Account - Benefits

- **Reduces manual effort:** Automates repetitive operational tasks
- **Improves consistency:** Ensures tasks run the same way every time
- **Enhances security:** Automated patching and secure credential storage
- **Supports hybrid environments:** Automate Azure and on-prem resources
- **Cost-efficient:** Pay only for job runtime; no servers to manage
- **Scalable automation:** Run tasks on demand or on schedule at any scale
- **Integrates with IaC:** Works seamlessly with ARM, Bicep, Terraform, and DevOps pipelines
- **Centralized governance:** Unified place to manage automation, logs, and compliance

85

Azure Automation Account - Workflow

- Create Automation Account
 - add managed Identity
- Assign permissions to managed Identity
- Create Runtime Environment
 - add/update Modules
- Create share resources (if necessary)
 - Variables, credentials, certificates
- Create Runbook
- Configure schedule and/or webhook
 - or start Runbook manually

86



GitHub & Azure DevOps



87

GitHub

GitHub is a cloud-based platform for hosting, managing, and collaborating on software projects using Git. It provides developers with tools for version control, code review, automation, documentation, and project management — all in one place. GitHub enables teams to work together efficiently, track changes, manage issues, and deliver high-quality software through a shared, centralized repository.

Key Benefits:

- **Version Control with Git:** Track changes, manage branches, and collaborate safely
- **Pull Requests & Code Reviews:** Structured collaboration and quality control
- **GitHub Actions:** Built-in CI/CD automation for testing, building, and deploying
- **Project Management Tools:** Issues, boards, milestones, and workflows
- **Open-Source Ecosystem:** Millions of public repositories and reusable packages
- **Documentation Support:** Wikis, README files, and integrated Markdown rendering
- **Cloud-Hosted:** No infrastructure to manage; accessible from anywhere
- **Community & Discoverability:** Enables sharing, learning, and contributing globally

88

Azure DevOps

Azure DevOps is Microsoft's end-to-end platform for software delivery and automation. For IaC, it provides a **centralized, controlled, and automated pipeline** for deploying cloud infrastructure using Bicep, ARM, Terraform, or Ansible.

Core IaC Capabilities

- **Azure Repos** Git-based version control for Bicep/ARM/Terraform code
Enables branching, pull requests, code reviews, and policy enforcement
- **Azure Pipelines** CI/CD automation for validating, planning, and deploying IaC
Supports YAML pipelines, multi-stage deployments, approvals, and environments
- **Service Connections** Secure authentication to Azure using Managed Identity or OIDC
Eliminates long-lived secrets in pipelines
- **Artifacts** Store reusable IaC modules, templates, or Terraform providers

89

Azure DevOps

Benefits

- **Repeatable deployments** with automated validation and testing
- **Governance & compliance** through approvals, policies, and audit trails
- **Modular IaC workflows** using templates, stages, and reusable pipeline components
- **Enterprise-grade security** with RBAC, OIDC, and protected branches
- **Supports multi-cloud** (Azure, AWS, GCP) and hybrid deployments

90

Azure DevOps versus GitHub

GitHub

- **Focus:** Open-source collaboration, modern DevOps, cloud-native workflows
- **Strengths:**
 - Best-in-class Git hosting
 - Pull Requests & code reviews
 - GitHub Actions for CI/CD automation
 - Massive open-source ecosystem
 - Simple, developer-friendly UI
- **Best For:**
 - Cloud-native teams
 - Open-source projects
 - Lightweight CI/CD pipelines
 - Developer-centric workflows

Azure DevOps

- **Focus:** Enterprise DevOps, project management, end-to-end delivery
- **Strengths:**
 - Azure Repos (Git)
 - Azure Pipelines (CI/CD)
 - Azure Boards (project & work tracking)
 - Azure Artifacts (package management)
 - Deep enterprise governance
- **Best For:**
 - Large organizations
 - Complex project management
 - Multi-team coordination
 - Hybrid or on-prem integration

91

GitHub - Features (not exhaustive)

Repositories

- Central place to store and manage code
- Supports Git version control
- Public or private
- Branching, merging, pull requests

GitHub Projects

- Kanban-style boards for planning and tracking work
- Integrates with Issues and Pull Requests

Issues

- Track bugs, tasks, enhancements
- Labels, milestones, assignees

Pull Requests (PRs)

- Code review and collaboration workflow

GitHub Actions

- Built-in CI/CD automation
- Automate builds, tests, deployments

Wikis

- Full documentation space per repository

Security Features

- Dependabot alerts & updates
- Secret & Code scanning
- Security advisories

Discussions

- Community Q&A and long-form conversations

Codespaces

- Cloud-hosted development environments
- VS Code-powered, instant setup

92

GitHub - Doing

- Learning Paths
 - <https://learn.microsoft.com/en-us/training/paths/github-foundations/>
 - <https://learn.microsoft.com/en-us/training/paths/github-foundations-2/>
 - <https://learn.microsoft.com/en-us/training/paths/github-administration-products/>
 - <https://learn.microsoft.com/en-us/training/paths/github-admin-2/github-admin-2>
 - <https://learn.microsoft.com/en-us/training/paths/github-actions/>
 - <https://learn.microsoft.com/en-us/training/paths/github-actions-2/>
- Create Profile
- Create Repository
- Push Code

93

Git - Remote Repository Authentication

- HTTPS + Personal Access Token (PAT)
 - used if HTTPS-Url is used
- PAT creation
 - GitHub: Settings -> Developer settings -> Personal Access Token
- Usage:

94

Git - Remote Repository

- To link a local repo to a remote one

```
git remote add origin '<url>'
git remote add -u origin '<url>'
```

- parameter '-u' let git link current to remote branch
- 'origin' is just a name of the remote repo-url (!)

- To push code and commits

```
git push origin main
```

- *'push local repo in current branch to the remote repo origin to the remote branch main'*
- only current branch is pushed
- if use '-u' while adding remote repo

```
git push
```

95

Git - Remote Repository

- to avoid of confusion of branch names rename branches
 - newly created GitHub repositories have the 'main' branch
 - newly create local repositories maybe 'master'

```
git remote add origin '<url>'
git remote -v
git branch -M main
git push -u origin main
```

- to push all branches of local repo

```
git push --all origin
```

96

Git - Remote Repository

- in case of working with existing remote repo, clone it

```
git clone '<url>'
```

- what is cloned?
 - all files, all branches
 - full commit history & tags
 - remote connections

- use cases:

```
# clone into custom folder
git clone '<url>' /path/to/folder
# clone specific branch
git clone -b feature/cba '<url>'
# clone without history
git clone --depth 1 '<url>'
# clone only single branch
git clone --single-branch --branch feature/cba '<url>'
# clone with SSH
git clone git@github.com:user/repo.git
```

97

Git - Remote Repository

- to get changes from remote to local repo

```
git pull origin main
git pull
```

- to see changes in remote repo without merging

```
git fetch
git fetch -v
git fetch --all
```

98

GitHub Actions – Automation & CI/CD

GitHub Actions is GitHub's built-in automation and CI/CD platform. It lets you run workflows directly from your repository in response to events like pushes, pull requests, releases, or schedules. With Actions, you can automate builds, tests, deployments, security checks, and any custom task using YAML-based workflows.

Key Capabilities:

- **CI/CD Pipelines:** Build, test, and deploy applications automatically
- **Event-Driven Automation:** Trigger workflows on push, PR, issue creation, schedules, or manual runs
- **Reusable Workflows:** Share and reuse workflow logic across repositories
- **Secrets & Secure Variables:** Store credentials safely for deployments
- **Self-Hosted Runners:** Run workflows on your own infrastructure
- **Marketplace Actions:** Thousands of prebuilt actions for cloud providers, testing tools, security scanners, and more

100

GitHub Actions

1. A workflow starts from an event

- push
- pull request
- schedule
- manual dispatch
- release

2. Actions splits the workflow into jobs

- Each job runs independently
- can run in parallel
- has its own runner

3. Runners execute steps

- A runner checks out code
- installs tools, runs scripts
- builds, tests, deploys
- uploads artifacts

4. Workflow completes when all jobs finish

- GitHub reports success,
- failure, skipped or cancelled

101

Deployment Stacks

Azure **Deployment Stacks** provide lifecycle management for IaC-deployed resources. They track, own, update, and delete resources created by a Bicep/ARM deployment — acting as a *management boundary* for everything deployed through the stack.

Why are they made?

- Traditional ARM/Bicep deployments don't "own" resources after creation
- Hard to clean up resources safely
- Drift is difficult to detect
- Multi-template environments become messy over time
- Deployment Stacks solve this by **binding resources to a stack**.

Key Capabilities

- **Resource Ownership** The stack tracks all resources created by the deployment.
- **Safe Deletion** Deleting the stack can delete all associated resources automatically.
- **Drift Detection** Identifies changes made outside IaC.
- **Scoped Updates** Updates only the resources managed by the stack.
- **Supports Bicep & ARM** Works with existing IaC workflows.

103

Deployment Stacks

Benefits

- Cleaner lifecycle management
- Predictable updates and deletions
- Reduced risk of orphaned resources
- Better governance for large environments
- Ideal for modular or multi-team IaC setups

104

Deployment Stacks - autoOnUnmanage

Controls what happens when a resource **falls out of management** (i.e., it exists in Azure but is no longer defined in the stack template).

Modes

- **delete** Automatically deletes unmanaged resources when the stack is updated
 - Ensures strict alignment with IaC.
- **detach** Removes the resource from the stack's ownership but *keeps it in Azure*.
 - Useful when resources should persist but no longer be managed by IaC.

105

Deployment Stacks - denySettingsMode

Controls how Azure handles **manual changes** to resources that are owned by the Deployment Stack.

Modes

- **none** Manual changes are allowed
 - *Least restrictive.*
- **denyDelete** Prevents deletion of stack-owned resources outside IaC
 - *Protects critical infrastructure.*
- **denyWriteAndDelete** Blocks *all* manual modifications and deletions
 - *Ensures full IaC immutability.*

106